

课程笔记

与 vLLM 团队畅聊多模态（下）

五道口纳什 & Codex

2026 年 7 月 17 日



视频作者/频道：月球大叔 / vLLM 社区

发布日期：2025 年 7 月

视频时长：31 分 32 秒

视频链接：<https://www.bilibili.com/video/BV1Vy5d6DE7B/>

目录

1	世界模型与扩散优化	2
1.1	从游戏渲染到世界模型：Focus Area 的思路	2
1.2	本章小结	2
2	框架设计：插件机制与并行策略	4
2.1	vLLM 多模态框架的设计哲学	4
2.2	CFG Parallel (CFP) 并行策略	4
2.3	PR 实验的算力需求与 Layer-wise 策略	4
2.4	本章小结	5
3	志鹏的贡献之路：从零基础到 Committer	6
3.1	起点：一个传统软件工程师的转型	6
3.2	第一步：找到并修复一个 Bug	6
3.3	从 Bug 修复到模型支持：找到自己的赛道	6
3.4	攻克 ViLBOK：从放弃到成功的迂回之路	7
3.5	从 Committer 到核心贡献者	7
3.6	本章小结	7
4	新人参与开源：降低门槛的实践建议	8
4.1	减小模型规模以降低算力门槛	8
4.2	「Code with AI」——新一代开发者的核心技能	8
4.3	Continuous Batching 在 Diffusion 中的挑战	8
4.4	本章小结	9
5	OMI 项目入门与学习路径	10
5.1	第一步：最小化跑通	10
5.2	Bug 选择的策略：从简单到复杂	10
5.3	学习节奏：周会 + 录播 + 关键词追踪	11
5.4	进阶：从单 Stage 到 Diffusion 模型	11
5.5	本章小结	11
6	单元测试与验证技术	12
6.1	AI 生成代码时代的验证挑战	12
6.2	End-to-End vs Unit Test 的权衡	12
6.3	Composition Testing：像烤肉串一样组合测试	12
6.4	本章小结	13
7	ViT 技术与视频帧优化	14
7.1	ViT 的当前格局与技术瓶颈	14
7.2	Patch-Level Attention：按需「关注」而非全量采样	14
7.3	「先识别，再处理」的两阶段策略	14
7.4	本章小结	15

8 异构硬件适配与社区生态	16
8.1 硬件异构带来的适配挑战	16
8.2 从个人贡献到系统构建：飞轮效应	16
8.3 参与开源的长远价值	16
8.4 本章小结	16
9 总结与延伸	17
9.1 讲者结尾讨论	17
9.2 核心主张与方法论提炼	17
9.3 贡献飞轮：从零到指数级增长	17
9.4 开放问题与后续方向	18
9.5 拓展阅读	18

1 世界模型与扩散优化

1.1 从游戏渲染到世界模型：Focus Area 的思路

本节讨论从游戏渲染的工程优化思路出发，理解世界模型（World Model）中扩散生成的计算效率问题。

游戏渲染中的 Focus Area 策略。游戏渲染需要在高帧率下保持流畅体验，但在 GPU 算力有限的年代，开发者采取了一种经典的优化策略：

- **Focus Area（焦点区域）：**角色快速奔跑时，画面中央（角色关注区域）精度极高。
- **非焦点区域：**画面两侧和上下区域的渲染质量明显降低，甚至模糊。
- **人类感知的容忍度：**由于处于快速运动中，玩家并不会察觉边缘区域的画质下降。

核心类比：从渲染到扩散

游戏渲染的 focus-area 策略揭示了计算资源分配的基本原则——**并非所有像素需要同等精度**。这一思想可以直接映射到扩散模型（Diffusion Model）的生成过程中：背景和边缘区域对细节的要求远低于前景和核心内容区域。

扩散模型中的步数优化。传统扩散模型通常需要约 50 步才能生成一张精细图像，但实际应用场景中并不总是需要完整质量：

- **频繁迭代场景：**当需要生成连续变化（如视频帧序列）时，每帧的细节并不需要达到“静态展示”级别。
- **步数压缩：**可以将步数从 50 步降至 25 步甚至更低——因为人类视觉系统在连续帧间主要关注变化区域，对静态背景的精度容忍度更高。
- **蒸馏加速：**工业界已通过蒸馏技术，将扩散步数压缩至 **5 步**，质量与原 50 步效果相当。

世界模型中的 Latent 预测

世界模型的一种重要形式并不是做像素级的图像生成，而是对未来的 Latent 表示做预测。例如 Google DeepMind 的 Genie 模型，它不去生成像素，而是在浅层潜在空间中对未来状态进行预测。这种方式计算代价更低，但需要社区协作来探索具体需求和应用方向。

单机分工：粗粒度 + 精细生成协同。在同一台机器上，可以同时运行两类生成模块：

- **粗粒度生成器：**快速生产背景和低关注度区域的图像。
- **精细生成器：**仅对焦点区域进行高质量生成。
- 这种分工策略类似于 GPU 渲染管线中的 LOD（Level of Detail）机制——根据关注的远近分配计算资源。

1.2 本章小结

世界模型的优化思路与游戏引擎的渲染优化高度相通。核心思想是**计算资源的差异化分配**——焦点区域高精度，边缘区域低精度。扩散模型的步数可以通过蒸馏大幅压缩（50 步 → 5 步），而

Latent-space 预测为世界模型提供了比像素生成更经济的技术路径。这些方向需要社区和工业界共同探索。

2 框架设计：插件机制与并行策略

2.1 vLLM 多模态框架的设计哲学

本段内容来自直播中的观众问答环节，涵盖了 vLLM 多模态框架的插件机制、CFP 并行策略、以及 PR 实验的算力需求。

插件机制现状。目前 vLLM 多模态框架并不提供正式的插件（Plugin）机制。然而这并非设计缺陷，而是框架架构特点所致：

- **框架灵活性：**与某些主仓框架不同，vLLM 多模态框架并非高度稳定、难以拆分的架构。许多功能可以直接接入使用，无需通过插件层。
- **低接入成本：**”你可以直接接进来就能用”——框架对直接集成持开放态度。

框架设计的两种哲学

高度模块化的框架（如某些主仓）需要插件机制来保证稳定性，因为核心逻辑难以改动。而 vLLM 多模态框架的设计理念是保持灵活——与其建立一个复杂的插件系统，不如保持代码的可直接修改性，让贡献者更容易上手。

2.2 CFG Parallel (CFP) 并行策略

CFP (CFG Parallel) 是 vLLM 中的一种并行推理策略：

- **基本机制：**当一个请求（Prompt）进入系统后，系统会生成一个空的 Prompt 作为该请求的派生（Companion）请求。
- **本质：**CFP 并非同时处理两个独立请求，而是对一个请求进行”分身”式并行——主请求与派生请求共享计算资源但处理不同的生成路径。
- **与 CFG 的关系：**CFP 源于 Classifier-Free Guidance (CFG)，通过对比有条件和无条件生成的输出来提升质量。

2.3 PR 实验的算力需求与 Layer-wise 策略

算力需求的核心原则

PR 实验的算力需求并非固定值，而是取决于模型规模和特性类型。对于新人贡献者，建议遵循的策略是：先用小模型验证特性，再扩展到生产规模。

算力分级指导：

- **Layer-wise 适配：**可以在单张卡甚至个人电脑上完成。只需验证”原来在单卡上跑不通的模型，加入 Layer-wise 支持后能跑通”即可。
- **基础 PR 实验：**最小配置通常需要 2 张 GPU。
- **复杂模型并行：**涉及模型并行（Model Parallelism）和数据并行（Data Parallelism）时，可能需要 4-8 张卡。

2.4 本章小结

vLLM 多模态框架选择了**灵活性优先于形式化插件**的设计路线，降低了社区贡献的门槛。CFP 是一种派生式并行策略，不同于简单的同时处理多个请求。算力需求应从小模型起步——Layer-wise 优化甚至可在个人电脑上验证。

3 志鹏的贡献之路：从零基础到 Committer

3.1 起点：一个传统软件工程师的转型

本节是整个直播中最有价值的部分之一。志鹏 (Zhipeng) 详细分享了他从 2024 年 4 月第一次接触 AI Infra 到成为 vLLM-OMNI 项目 Committer 的全过程。

初始背景：

- 2024 年 4 月之前：做传统软件开发（前后端），完全没听说过 vLLM。
- 第一次接触 AI Infra：通过观看”大叔”和”一华芬太”的 B 站直播。
- 行业判断：”传统软件行业基本上就完蛋了，AI 太牛逼了，可能过几个月我就要失业了”——这一认知驱使他主动转型。

从零开始的现实

志鹏第一次看直播时，完全听不懂——”充斥各种行业黑话”。他不是天才型入门者，而是通过持续的实践和社区互动逐步成长。**起点低不是障碍，持续投入才是关键。**

3.2 第一步：找到并修复一个 Bug

Bug 追踪策略：

1. 发现一个与 CPU 相关的 bug。
2. 逐版本追溯：从 v0.8.0 往前翻到 v0.6.x，锁定引入 bug 的具体版本。
3. 尝试自行修复但能力不足，主动向社区成员（凯超 / Kai Chao）求助。
4. 在交流中意识到：vLLM 太火了，连修 bug 都有人”抢”——提一个 Issue 后别人会迅速提交修复。

3.3 从 Bug 修复到模型支持：找到自己的赛道

关键决策：志鹏注意到社区强调”很多模型需要做支持 (Model Support)”。由于 VLM 领域火热的竞争让他抢不到 bug，他选择了一个差异化方向——**做多模态模型的支持 (Model Integration)**。

第一个模型：TERA3CY (字节跳动开源模型)：

- 耗时：约一个月，从看文档到跑通。
- PR 规模：约 1000 行代码。
- Review 过程：莫子峰 (Mo Zifeng) 和 Cyrus 联合审阅，PR 上积累了 70-80 条 review 记录。
- 学到的内容：框架整体理解、代码规范 (coding style)、PR 提交注意事项。

贡献与学习的辩证关系

”你对于仓库做出来的贡献，是要远小于你真正在贡献中学到的东西的。”这是志鹏分享的核心洞见。vLLM 和 vLLM-OMNI 社区的 review 文化非常友好，资深成员愿意花时间指导新人——这种指导本身比代码改动更有价值。

3.4 攻克 ViLBOK：从放弃到成功的迂回之路

第三个模型的挑战——ViLBOK：

- 独特之处：ViLBOK 是一个需要改动 ViT (Vision Transformer) 的模型，与常规模型接入完全不同。
- 第一次尝试 (2-3 周)：因源代码极其复杂且没有 Transformer 版本，移植困难，无进展。
- 迂回策略：放弃 ViLBOK，先做 3-4 个其他模型积累经验。
- 重新定义任务：与 Roger (ViT 模块的技术负责人) 沟通后，确定只需在主仓实现 AR (Auto-Regressive) 部分。
- 最终方案：主仓实现 AR 部分，OMNI 仓库实现 Diffusion 部分。

社区角色的发现

志鹏在第三个模型时才“发现”Roger 才是 ViT 模块的技术负责人 (而非之前常接触的 Cyrus 或莫子峰)。**理解社区的“人员地图”需要时间**——知道谁的领域是什么、找谁求助最高效，这本身就是一种隐性知识。

3.5 从 Committer 到核心贡献者

完成 ViLBOK 的接入后，志鹏逐步参与了更多核心重构工作：

- Diffusion Worker 和 Diffusion Runner 的重构
- 调度器 (Scheduler) 模块的重构
- OMNI Connect 模块的重构
- 在两大仓库中实现了 ViLBOK 模型的完整接入

成长路径总结：Bug 修复 → 模型接入 (7-8 个模型) → 核心模块重构 → Committer

3.6 本章小结

志鹏的经历展示了一条清晰的**从零到 Committer 的路径**：从修 bug 入门 → 找到差异化方向 (模型支持) → 通过大量 review 反馈学习 → 攻克技术难题 → 参与重构 → 成为核心成员。整个过程的核心动力不是“天赋”，而是**持续的实践投入和社区的 mentorship 文化**。

4 新人参与开源：降低门槛的实践建议

4.1 减小模型规模以降低算力门槛

对于没有大量 GPU 的学生和个人开发者，社区提供了多种降低门槛的策略：

- **使用小模型变体**：例如 Qwen TTS 有 0.6B 的 variant，在小卡上完全可以运行。
- **语音模型更轻量**：大部分语音合成模型只有 2-3B 级别，远轻于视频生成和图像合成模型。
- **聚焦特定领域**：从语音类模型入手，算力需求更低，同样能学习框架的接入流程。

4.2 「Code with AI」——新一代开发者的核心技能

本节讨论了 AI 辅助编程对开发者角色的重塑。

编程的两个维度

编程本质上包含两个层面：

1. **实体层面**：写出具体的代码（Code as Text）。这一层面正在被 AI 替代。
2. **认知层面**：在脑海中建立对整个软件架构的认知模型，并利用这个模型做出正确的设计决策。这一层面暂时还无法被 AI 取代。

新一代开发者的技能要求：

- 能够通过 Prompt 指导 AI Agent 解决相对复杂的算法或系统问题。
- 能够识别系统中的 Bug，并用自然语言描述问题、引导 AI 定位。
- 核心能力从”手写代码”转变为”设计验证策略 + 评估 AI 产出”。

警惕表面繁荣

工作本身并没有消失，但对工作的要求在剧烈迭代。能熟练使用 AI 工具的人会获得指数级的效率提升；拒绝适应的人则面临被边缘化的风险。

4.3 Continuous Batching 在 Diffusion 中的挑战

问题背景：Continuous Batching 在 LLM 推理中已广泛使用，但在 Diffusion 模型中面临精度挑战。

- **历史经验**：早在 ChatGPT 出现之前，Diffusion 领域就尝试过 Batching，但效果不佳。
- **质量退化**：不同 Step 的 Layer 在做并行时会产生质量退化（Quality Degradation），因为不同步骤之间的去噪程度不同，直接混合会导致精度损失。
- **当前状态**：已有公司在尝试解决这个问题，vLLM 社区也在关注相关进展。”摸石头过河”——参考先行者的经验。

研究机会：Diffusion Batching

这是一个尚未收敛的研究方向。如果你能证明 Diffusion Batching 在精度可接受的前提下是可行的，这就是一篇高质量的论文机会。多模态领域”研究问题到处都是”，正是学术界参与的最佳时机。

4.4 本章小结

新人参与开源的核心策略是**降低算力门槛**（用小模型）和**提升 AI 协作技能**（用 Prompt 而非手写代码解决问题）。在 AI 时代，人类开发者的独特价值在于**建立系统的认知模型**和**做出设计决策**。Diffusion Batching 的精度问题是一个开放的研究方向。

5 OMI 项目入门与学习路径

5.1 第一步：最小化跑通

志鹏分享了带领新加坡学习小组参与 OMI (vLLM-OMNI) 项目的实践经验，给出了可操作的入门路径：

1. **选最小的模型跑通 vLLM-OMNI**：先让整个流程动起来。
2. **阅读文档**：每个模型的 README 都有使用说明。
3. **跑通单元测试**：验证环境配置正确。
4. **浏览 Issue 列表**：寻找可修复的 Bug。



图 1: 志鹏分享 OMI 项目入门经验¹

5.2 Bug 选择的策略：从简单到复杂

Model-level Bug (推荐入门)：

- 某个模型输出全是噪声 (Noise)。
- 某个模型以前能跑通，现在跑不通了——说明是回归 Bug。
- 这类 Bug 通常范围明确，修复路径清晰。

框架级 Bug (进阶)：

- 涉及调度器、内存管理、并行策略等核心模块。
- 需要更深的框架理解，不建议作为第一个 PR。

¹视频画面时间区间：00:20:00-00:23:00。

5.3 学习节奏：周会 + 录播 + 关键词追踪

社区学习机制：

- 每周三 11:30 (北京时间)：vLLM-OMNI 社区周会。
- 参与方式：第一次可能完全听不懂——这是正常的。记下所有听不懂的关键词。
- 会后学习：利用录播回看 + 搜索关键词，逐步建立术语体系。
- 迭代理解：每周积累几个新概念，几周后就能跟上讨论节奏。

5.4 进阶：从单 Stage 到 Diffusion 模型

- 第一阶段：接单 Stage 的多模态模型（如简单的 VLM）。
- 第二阶段：尝试 Diffusion 类模型，这类模型需要理解更复杂的生成流程。
- 并行学习：在此过程中自然接触到 embarrassingly parallel、模型并行、数据并行等分布式概念。

从小白到跑通模型只需一个下午

志鹏的经验：如果专注投入，”从完全不了解 vLLM-OMNI 到跑通第一个模型，一个下午就够了”。关键在于 **不要试图一次性理解整个框架**——先让代码跑起来，再逐步理解每个模块。

5.5 本章小结

OMI 项目的入门路径：跑通最小模型 → 阅读文档 + 单测 → 修 Model-level Bug → 参加周会积累术语 → 逐步挑战更复杂的模型和框架模块。核心原则是**渐进式深入**，先实践再理论。

6 单元测试与验证技术

6.1 AI 生成代码时代的验证挑战

随着 AI 辅助编程的普及，社区中出现了大量 AI 生成的测试代码。这带来了一个新的质量问题：

AI 生成测试的隐患

社区中许多 PR 的测试代码是 AI 生成的，但这些测试往往缺乏真正的验证价值。**AI 能写代码，但不能思考”这段代码是否正确”**。验证的责任落在了人类开发者身上。

算子的验证挑战：

- 很多公司已经用 AI 生成 CUDA 算子代码。
- 人类的主要工作时间从”写算子”转变为”思考如何验证算子”。
- 验证需要：理解上下文、知道如何证明正确性、确定哪些路径需要覆盖。

6.2 End-to-End vs Unit Test 的权衡

在 GPU 资源昂贵的前提下，测试策略的选择直接影响 CI 效率：

- **End-to-End Test**：重构代码后容易大面积崩溃，维护成本高。但覆盖了组件间的交互，能发现集成层面的问题。
- **Unit Test**：相对稳定，代码重构时不易受影响。但难以覆盖跨模块的”化学反应”。
- **OMI 的 5 级测试体系**：按照算力消耗和权重下载成本，将测试分为 5 个级别，不同级别的触发时机不同。

6.3 Composition Testing：像烤肉串一样组合测试

这一节提出了一个非常形象的测试策略概念：

Composition Testing 的核心思想

在有限的 GPU 资源下，你需要**像穿烤肉串一样**把最重要的组件串在一起测一遍。不同组件的 Composition 之间会产生”化学反应”（Chemical Reaction）——你需要精心设计”穿多少根签子、引多少根线”才能用最少资源覆盖最多路径。

实际挑战：团队可能只有有限的几张 GPU，但有 20-30 个 Kernel 级测试需要跑。如何做 Exhaustive Testing 并确定组件在不同组合下都能正常工作，是一个需要精心设计的技术活。



图 2: 讨论测试策略时的直播画面²

6.4 本章小结

在 GPU 资源昂贵且 AI 代码生成普及的时代，**验证 (Verification) 比生成 (Generation) 更重要**。OMI 的 5 级测试体系通过分级触发来平衡覆盖率和成本。Composition Testing 的”烤肉串”策略——用最少资源覆盖最多组件交互——是值得所有 GPU 密集型项目借鉴的思路。

²视频画面时间区间：00:24:00–00:27:00。

7 ViT 技术与视频帧优化

7.1 ViT 的当前格局与技术瓶颈

本节讨论 Vision Transformer (ViT) 在多模态模型中的应用现状和优化方向。

当前 ViT 的主流方案：

- 基于 ViT / CLIP 的架构仍是主流。
- 视频处理时通常每秒取 1-2 帧。
- **问题：**这样的采样策略会消耗大量 Token——每帧经 ViT 编码后产生数百个 Token，长视频的 Token 总数迅速膨胀。

7.2 Patch-Level Attention：按需「关注」而非全量采样

讨论中提到了一篇新的研究思路：

- **传统方式：**提取视频的每一帧、对每一帧的每个 Patch 做编码。
- **新思路：**在输入端做 Patch 级别的注意力 (Attention)，**不提取每个 Patch**——只提取局部关键区域的信息。
- **效果：**对输入信息进行压缩，减少无用 Token 的数量。

7.3 「先识别，再处理」的两阶段策略

受坤泰 (Kuntai) "C-Code / D-Script" 思路启发，团队提出了一种两阶段策略：

1. **Stage 1：小模型识别**——用一个轻量级模型快速扫描视频，识别哪些帧/区域包含重要信息。
2. **Stage 2：大模型精处理**——用高精度模型仅处理 Stage 1 标记的关键区域。

新研究方向：Range of Interest for World Models

直播中提出了一个新的概念——**世界模型的兴趣区域 (Range of Interest / RoI for World Models)**。对于世界模型而言，并不需要关注视频的每一个细节。一个高层次的“概览”（大致知道发生了什么）可能就足够了。这意味着可以在信息压缩上比传统 VLM 走得更远——从“压缩语义”到“压缩感知范围”。



图 3: ViT 与视频帧优化讨论³

7.4 本章小结

ViT 在视频多模态处理中的核心瓶颈是 Token 消耗过大。解决思路包括：**Patch-Level Attention**（按需提取而非全量采样）和**两阶段策略**（小模型识别 + 大模型处理）。「Range of Interest for World Models」作为一个新概念，提示了世界模型可以从”压缩信息”进一步走向”压缩感知范围”——这为学术界提供了新的论文方向。

³视频画面时间区间：00:28:00–00:29:30。

8 异构硬件适配与社区生态

8.1 硬件异构带来的适配挑战

直播最后讨论了 vLLM 社区面临的一个现实问题——硬件异构性：

- **现状**：许多公司基于 vLLM 做了内部 Fork，在其中加入大量针对特定硬件（如国产芯片）的 Hard-code 适配。
- **困难**：这些 Fork 与上游主线的适配逻辑差异大，合并回社区的难度很高。
- **长期维护问题**：上游更新后，Fork 需要持续追主线，维护成本随时间累积。

8.2 从个人贡献到系统构建：飞轮效应

志鹏分享了他当前的思考方向——从写代码转向“构建能写代码的系统”：

贡献的指数级增长

“当你把这东西 Build 出来以后，你的贡献会是指数级的上升。”与其持续手写代码解决单个问题，不如投入时间构建一套能自动解决问题的反馈系统。这类似于建立了一个“飞轮”——一旦转起来，效率和贡献量都会超过线性增长。

「飞轮」的核心组件：

- **验证机制**：Roger 反复强调的“怎么验证”——验证做好了，轮子就能转。
- **反馈闭环**：自动检测 → 自动定位 → 自动修复 → 自动验证。
- **技能积累**：推荐 oxytra traitor 等 research 站点的学习资源，用于系统性提升工程能力。

8.3 参与开源的长远价值

开源贡献的复利效应

从修 Bug 开始 → 逐步参与核心开发 → 成为 Reviewer / Committer → 影响项目方向。每一步都建立在之前积累的信任和能力之上。即使一开始的贡献量很小，持续参与带来的网络效应（认识的人、积累的知识、建立的声誉）会产生复利式的增长。

8.4 本章小结

异构硬件适配是 vLLM 社区面临的长尾挑战，需要从 Fork 合并机制和平台抽象两个层面解决。在个人发展层面，从“写代码的人”进化为“构建系统的人”是贡献量从线性到指数级跃升的关键。建立验证-反馈闭环，让飞轮转起来，比一个一个修问题更有长远价值。

9 总结与延伸

9.1 讲者结尾讨论

直播最后，几位讲者围绕“如何让贡献飞轮转起来”进行了开放讨论。核心观点包括：

- **验证是飞轮的核心**：Roger 反复强调的“验证问题”——只有建立了可靠的验证机制，代码生成、自动修复等后续环节才能自动运转。
- **从个人贡献到系统构建**：志鹏表示他正在学习如何“Build 出能写代码的系统”，而非继续逐个手写 PR。这是一个从线性贡献到指数级贡献的思维跃迁。
- **开源社区的长期价值**：对于异构硬件、社区 Long-term Support 等挑战，需要社区层面建立可持续的协作机制，而非依赖个别公司的内部 Fork。

9.2 核心主张与方法论提炼

本场讨论围绕一条主线展开：**如何在 AI Infra 时代实现个人的成长与社区的繁荣**。以下是从讨论中提炼的五条核心主张：

1. **计算资源的最优分配**（对标世界模型优化）。世界模型中 Focus Area 的思路——差异化分配计算资源——不仅是渲染/生成的优化策略，也是一种通用的工程思维。在有限资源下（GPU、时间、注意力），**区分核心与边缘、精确与粗糙**，是高效工作的基本法则。
2. **从零开始的可复制路径**。志鹏的经历证明：不需要 AI 背景、不需要顶级学历、不需要大量 GPU。只要持续投入 + 利用社区资源（直播、周会、Review 反馈），一条从 **Bug Fix 到 Committer** 的路径是可复制、可验证的。
3. **AI 时代的人类角色**。编程正在被重新定义：实体代码的编写被 AI 替代，但 **建立认知模型 + 做出设计决策 + 设计验证策略** 的能力变得更加珍贵。这不是人类的退场，而是角色的升维。
4. **验证驱动的飞轮**。从测试策略（OMI 5 级体系）到 Composition Testing（烤肉串策略），再到志鹏“Build 出能写代码的系统”的目标——所有这些都指向同一个核心：**验证是自动化的前提，自动化是飞轮的基础**。
5. **多模态的结构性机会**。与大语言模型（结构已趋于稳定、由工业界主导）不同，多模态模型的结构尚未收敛。这意味着**学术界的 Idea 有更大的可能直接转化为社区的工程实现**——这是一个“遍地是研究问题”的窗口期。

9.3 贡献飞轮：从零到指数级增长

下面的 TikZ 图展示了开源贡献者从入门到指数级成长的典型路径。图中的每个阶段都对应本场讨论中的具体实践：

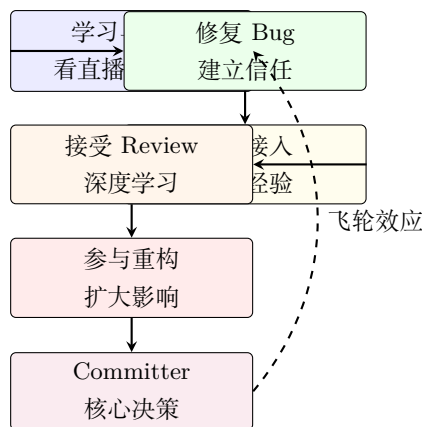


图 4: 开源贡献者的成长飞轮：从学习观察到核心决策的六个阶段

飞轮的动力来源：

- **外部动力：**社区的 Mentorship 文化（Review 反馈、周会指导、录播资源）。
- **内部动力：**持续实践带来的技能积累和领域理解的深化。
- **加速因素：**找到差异化方向（如模型支持）、建立验证闭环、从个人贡献转向系统构建。

9.4 开放问题与后续方向

基于本场讨论，以下问题值得进一步探索：

- **Diffusion Batching 的精度-效率权衡：**在多大程度上可以在扩散模型中安全地使用 Continuous Batching？是否可以通过 Step-aware Scheduling 来解决不同 Step 之间的精度冲突？
- **Patch-Level ViT 的工程实现：**现在还没有模型将 Patch-Level Attention 用作视频编码（Video Encoder）——这个方向的实现难度和效果提升如何？
- **世界模型的 RoI (Region of Interest)：**是否可以设计一种机制，让世界模型”主动决定”哪些区域需要高精度关注、哪些区域可以用粗粒度处理？
- **异构硬件的社区适配模式：**如何设计一种抽象层，让上游仓库不依赖具体硬件，但各硬件厂商可以独立维护适配插件？
- **AI Infra 的新人培养体系：**能否将志鹏的经验（修 Bug → 模型接入 → 重构 → Committer）形式化为一条可教学的路径？

9.5 拓展阅读

- vLLM GitHub ——主仓库，包含 Language Model 的推理引擎。
- vLLM-OMNI GitHub ——多模态扩展仓库，涵盖 Diffusion Model 和各种 VLM 的支持。
- Google DeepMind Genie ——基于 Latent-space 预测的世界模型（文中讨论的”非像素生成”世界模型范例）。
- ”Range of Interest” in Video Understanding ——视频理解中基于注意力机制的区域选择性处理方向。